

RUN — Building SlotBook live with the Virtual Office

Prereq: the office exists (`.claude/agents/*.md` from the demo runbook) and you are inside `virtual-office-demo/`. Copy this kit's `tickets/` into the repo.

```
cp -r booking-app-kit/tickets/* tickets/    # spec, design, SDLC, T-1..T-5
```

```
git add -A && git commit -qm "load SlotBook spec + tickets"
```

You have two ways to run the build. Pick before the talk.

Option A — Reliable (pre-baked spec/tickets, recommended for a live stage)

Skip generation; go straight to the plan + build.

@cto Read tickets/SPEC.md, tickets/DESIGN.md and tickets/T-*.md. Post a delegation

plan as a table (owner role + parallel/sequential) following tickets/SDLC.md.

Treat the API contract in DESIGN.md as frozen.

Then execute (see "Execute the build" below).

Option B — Impressive (generate the spec live, then build)

Let the PM and Architect produce the artifacts on stage from a one-liner.

@pm Feature request: SlotBook — a small appointment booking app for a barber.

Customers pick a provider, a service, a date, and an open 30-min slot (09:00–17:00)

and book it with their name and an optional note. Staff get a front-desk list of a

day's bookings. In-memory only, no auth/DB. Write SPEC.md and the tickets.

@architect Read tickets/SPEC.md and write tickets/DESIGN.md: stack, file layout,

data shapes, and the HTTP API contract. Freeze the contract so backend and frontend

can build in parallel.

(If you want the room to compare, diff the agents' output against this kit's versions.)

Execute the build

@cto Execute tickets/SDLC.md.

Step 1 (sequential): backend-dev does T-1 (store + slots) and its tests must pass.

Step 2 (parallel): launch backend-dev on T-2, backend-dev on T-3, and frontend-dev on T-4 — three workers, each in its own worktree. They code to the DESIGN.md contract.

Then qa-reviewer reviews each branch. Stop before merge for a human review.

Worktrees (if you isolate branches yourself):

```
git worktree add ../sb-T1 -b T1-store
```

```
git worktree add ../sb-T2 -b T2-availability
```

```
git worktree add ../sb-T3 -b T3-booking
```

```
git worktree add ../sb-T4 -b T4-page
```

The QA beat to hit on stage

@qa-reviewer Review the T3-booking branch. Confirm POST /api/bookings calls

isAvailable() and returns 409 on a repeat booking. Report blockers with file:line.

If a dev forgot the guard, bounce it back — that's your live evaluator–optimizer loop:

@backend-dev Fix the blocker qa-reviewer found on T3-booking (missing double-booking guard), add a test for it, then stop for re-review.

Integrate, run, and smoke-test

```
git checkout main
```

```
git merge --no-ff T1-store T2-availability T3-booking T4-page
```

```
node src/server.js      # serves on http://localhost:3000
```

```
# new terminal — prove the headline rule live
```

```
curl localhost:3000/api/services
```

```
curl "localhost:3000/api/availability?date=2026-07-01&providerId=p1"
```

```
curl -s -X POST localhost:3000/api/bookings -H 'content-type: application/json' \
```

```
-d
```

```
'{"date":"2026-07-01","time":"09:00","serviceId":"s1","providerId":"p1","customerName":"Test"}'
```

```
# run the SAME POST again -> expect HTTP 409 slot_taken (the money moment)
```

Then T-5 (sequential): @cto T-5 (front-desk view) depends on the bookings API. Run it now.

Hand-off into the AI-security blocks (4 & 5)

SlotBook already has the untrusted-input vector you need: the booking **note** field. For the breach demo, a "customer" submits a booking whose **note** carries an injection, and later someone asks an agent to "summarize today's bookings for the front desk." Reuse the poisoned-payload / guard.sh / permissions steps from the runbook, pointing the read at bookings instead of **tickets/T-9**. Same lethal-trifecta lesson, same fix.

```
@pm Summarize today's bookings for provider p1 for the front desk.
```

(With a permissive setup + an injected note, watch the exfil attempt; then apply the Block 5 permissions/hook and re-run — blocked.)